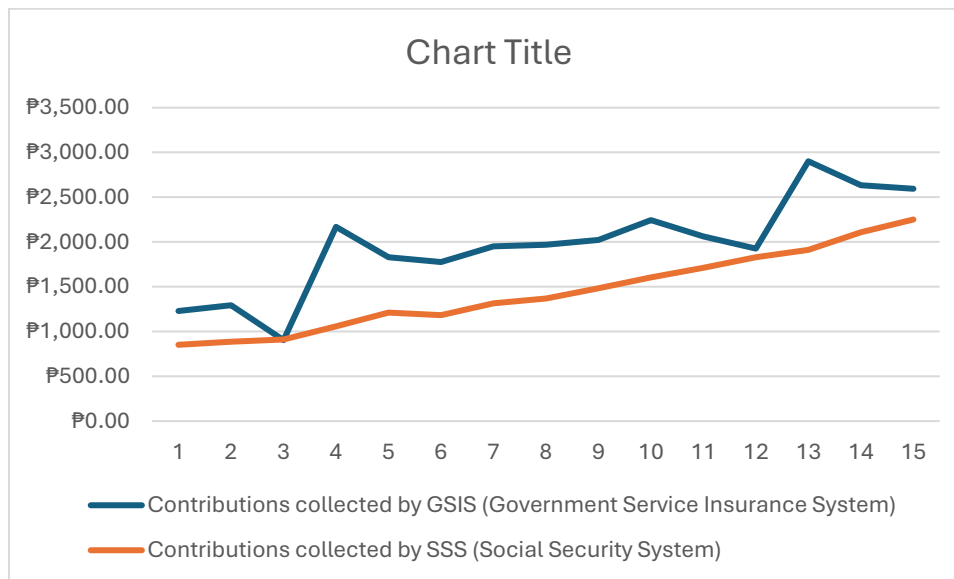


Reference:

Open data Philippines. (n.d.).

<https://data.gov.ph/index/public/dataset/Compendium%20of%20Employees'%20Compensation%20%28EC%29%20Statistics/tvb965di-8d3e-ay0w-9m4p-xrla2ngt5th5>

Year 2004 - up to date(x)	Contributions collected by GSIS (Government	Contributions collected by SSS (Social Security System)
1	₱1,228.16	₱852.24
2	₱1,292.49	₱887.12
3	₱904.03	₱910.21
4	₱2,169.20	₱1,059.60
5	₱1,831.50	₱1,211.08
6	₱1,775.49	₱1,183.94
7	₱1,951.88	₱1,315.87
8	₱1,969.64	₱1,368.60
9	₱2,020.97	₱1,482.50
10	₱2,243.64	₱1,605.13
11	₱2,060.61	₱1,711.20
12	₱1,924.67	₱1,828.07
13	₱2,901.18	₱1,913.62
14	₱2,632.09	₱2,107.49
15	₱2,592.80	₱2,251.17
20	₱3,097.95	₱2,746.08



```

import numpy as np

import matplotlib.pyplot as plt

import openpyxl

from pathlib import Path


# =====

# 1. FILE SETTINGS

# =====


# Get the folder where this Python script is located
SCRIPT_DIR = Path(__file__).resolve().parent


# Name of the Excel file
EXCEL_FILE = SCRIPT_DIR / "Book1.xlsx"


# Name of the worksheet
SHEET_NAME = "Sheet1"


# =====

# 2. READ THE EXCEL FILE

# =====


# Load workbook
workbook = openpyxl.load_workbook(
    EXCEL_FILE,
    data_only=True

```

)

Select worksheet

worksheet = workbook[SHEET_NAME]

=====

3. CREATE LISTS FOR THE DATA

=====

Independent variable

years = []

Dependent variables

gsis_contributions = []

sss_contributions = []

=====

4. READ DATA FROM EXCEL

=====

Assumed Excel columns:

#

Column A = Year

Column B = GSIS Contributions

Column C = SSS Contributions

#

```
# Row 1 contains the column headings.
```

```
# Data starts at Row 2.
```

```
for row in range(2, worksheet.max_row + 1):
```

```
    year = worksheet.cell(row=row, column=1).value
```

```
    gsis = worksheet.cell(row=row, column=2).value
```

```
    sss = worksheet.cell(row=row, column=3).value
```

```
# Only use rows containing valid values
```

```
if (
```

```
    year is not None
```

```
    and gsis is not None
```

```
    and sss is not None
```

```
):
```

```
    years.append(float(year))
```

```
    gsis_contributions.append(float(gsis))
```

```
    sss_contributions.append(float(sss))
```

```
# =====
```

```
# 5. CONVERT DATA TO NUMPY ARRAYS
```

```
# =====
```

```
x = np.array(years)
```

```
y_gsis = np.array(gsis_contributions)
```

```
y_sss = np.array(sss_contributions)
```

```
# =====
```

```
# 6. FUNCTION FOR LEAST SQUARES REGRESSION
```

```
# =====
```

```
def least_squares_regression(x, y, name):
```

```
    """
```

```
    Performs simple linear regression using  
    the least squares method.
```

```
    Model:
```

$$y = a_0 + a_1 \cdot x$$

```
    Parameters:
```

```
    x  = independent variable
```

```
    y  = dependent variable
```

```
    name = name of dataset
```

```
    Returns:
```

```
    a0
```

```
    a1
```

```

y_predicted
residuals
SSE
R_squared
"""

# -----
# Number of observations
# -----

n = len(x)

# -----
# Required sums
# -----

sum_x = np.sum(x)

sum_y = np.sum(y)

sum_xy = np.sum(x * y)

sum_x_squared = np.sum(x ** 2)

# -----
# Calculate slope a1

```

```
#
#       $n\sum xy - (\sum x)(\sum y)$ 
# a1 = -----
#       $n\sum x^2 - (\sum x)^2$ 
# -----
```

```
numerator = n * sum_xy - sum_x * sum_y
```

```
denominator = (
    n * sum_x_squared
    - sum_x ** 2
)
```

```
a1 = numerator / denominator
```

```
# -----
# Calculate intercept a0
#
#       $a0 = (\sum y - a1\sum x) / n$ 
# -----
```

```
a0 = (
    sum_y
    - a1 * sum_x
) / n
```

```
# -----
```

```
# Calculate predicted values
```

```
#
```

```
# y_hat = a0 + a1*x
```

```
# -----
```

```
y_predicted = a0 + a1 * x
```

```
# -----
```

```
# Calculate residuals
```

```
#
```

```
# Residual = Actual - Predicted
```

```
# -----
```

```
residuals = y - y_predicted
```

```
# -----
```

```
# Calculate Sum of Squared Errors
```

```
#
```

```
# SSE =  $\sum (y - y_{\text{hat}})^2$ 
```

```
# -----
```

```
SSE = np.sum(residuals ** 2)
```

```
# -----
```

```
# Calculate R-squared
```



```

#
#    $R^2 = 1 - \text{SSE} / \text{SST}$ 
# -----

y_mean = np.mean(y)

SST = np.sum(
    (y - y_mean) ** 2
)

R_squared = 1 - (SSE / SST)

# =====
# PRINT RESULTS
# =====

print()
print("=====")
print(f" LEAST SQUARES REGRESSION: {name}")
print("=====")

print()
print(f"Number of observations (n) = {n}")

print()
print("SUMS USED IN THE CALCULATION")

```

```
print("-----")
```

```
print(f"Sum of x    = {sum_x:.4f}")
```

```
print(f"Sum of y    = {sum_y:.4f}")
```

```
print(f"Sum of x*y   = {sum_xy:.4f}")
```

```
print(f"Sum of x^2   = {sum_x_squared:.4f}")
```

```
print()
```

```
print("REGRESSION COEFFICIENTS")
```

```
print("-----")
```

```
print(f"a0 (Intercept) = {a0:.6f}")
```

```
print(f"a1 (Slope)    = {a1:.6f}")
```

```
print()
```

```
print("REGRESSION EQUATION")
```

```
print("-----")
```

```
print(
```

```
    f"y = {a0:.6f} + ({a1:.6f})x"
```

```
)
```

```
print()
```

```
print("ERROR ANALYSIS")
```

```
print("-----")
```

```
print(f"Sum of Squared Errors (SSE) = {SSE:.6f}")
```

```
print(f"R-squared ( $R^2$ ) = {R_squared:.6f}")
```

```
# -----
```

```
# Return results
```

```
# -----
```

```
return (  
    a0,  
    a1,  
    y_predicted,  
    residuals,  
    SSE,  
    R_squared  
)
```

```
# =====
```

```
# 7. ANALYZE GSIS DATA
```

```
# =====
```

```
(  
    a0_gsis,  
    a1_gsis,  
    y_predicted_gsis,
```

```

    residuals_gsis,
    SSE_gsis,
    R2_gsis

) = least_squares_regression(
    x,
    y_gsis,
    "GSIS CONTRIBUTIONS"
)

# =====

# 8. ANALYZE SSS DATA

# =====

(
    a0_sss,
    a1_sss,
    y_predicted_sss,
    residuals_sss,
    SSE_sss,
    R2_sss

) = least_squares_regression(
    x,
    y_sss,
    "SSS CONTRIBUTIONS"

```

```
)
```

```
# =====
```

```
# 9. CREATE SMOOTH X VALUES FOR REGRESSION LINES
```

```
# =====
```

```
x_line = np.linspace(
```

```
    min(x),
```

```
    max(x),
```

```
    200
```

```
)
```

```
# =====
```

```
# 10. CALCULATE REGRESSION LINES
```

```
# =====
```

```
# GSIS regression line
```

```
y_line_gsis = (
```

```
    a0_gsis
```

```
    + a1_gsis * x_line
```

```
)
```

```
# SSS regression line
```

```
y_line_sss = (
```

```
    a0_sss
```

```
    + a1_sss * x_line
```

```
)
```

```
# =====
```

```
# 11. CREATE THE GRAPH
```

```
# =====
```

```
plt.figure(  
    figsize=(12, 7)  
)
```

```
# -----
```

```
# Plot actual GSIS data
```

```
# -----
```

```
plt.scatter(  
    x,  
    y_gsis,  
    s=70,  
    marker="o",  
    label="Actual GSIS"  
)
```

```
# -----
```

```
# Plot GSIS regression line
```

```
# -----
```

```

plt.plot(
    x_line,
    y_line_gsis,
    linewidth=2.5,
    label=(
        f"GSIS Regression: "
        f"y = {a0_gsis:.2f} + ({a1_gsis:.2f})x"
    )
)

# -----
# Plot actual SSS data
# -----

plt.scatter(
    x,
    y_sss,
    s=70,
    marker="s",
    label="Actual SSS"
)

# -----
# Plot SSS regression line
# -----

```

```

plt.plot(
    x_line,
    y_line_sss,
    linewidth=2.5,
    label=(
        f"SSS Regression: "
        f"y = {a0_sss:.2f} + ({a1_sss:.2f})x"
    )
)

# =====

# 12. ADD GRAPH LABELS

# =====

plt.title(
    "Simple Linear Regression by Least Squares\n"
    "GSIS and SSS Contributions vs Year",
    fontsize=14
)

plt.xlabel(
    "Year",
    fontsize=12
)

plt.ylabel(

```



```

    "Contributions Collected",
    fontsize=12
)

# Add grid
plt.grid(
    True,
    alpha=0.3
)

# Add legend
plt.legend(
    fontsize=9
)

# Adjust layout
plt.tight_layout()

# =====

# 13. SHOW THE GRAPH

# =====

plt.show()

# =====

# 14. FINAL COMPARISON

```

```
# =====
```

```
print()
```

```
print()
```

```
print("=====")
```

```
print(" FINAL COMPARISON: GSIS vs SSS")
```

```
print("=====")
```

```
print()
```

```
print("GSIS:")
```

```
print(
```

```
    f"Regression Equation = "
```

```
    f"y = {a0_gsis:.6f} + ({a1_gsis:.6f})x"
```

```
)
```

```
print(
```

```
    f"Slope (a1) = {a1_gsis:.6f}"
```

```
)
```

```
print(
```

```
    f" $R^2 = {R2\_gsis:.6f}$ "
```

```
)
```

```
print()
```

```
print("SSS:")
```

```
print(
```

```

f"Regression Equation = "
f"y = {a0_sss:.6f} + ({a1_sss:.6f})x"
)
print(
    f"Slope (a1) = {a1_sss:.6f}"
)
print(
    f"R2 = {R2_sss:.6f}"
)

print()

print("INTERPRETATION:")
print("-----")

if a1_gsis > a1_sss:

    print(
        "GSIS has a larger regression slope than SSS."
    )

    print(
        "This means GSIS contributions increase faster "
        "per year based on the fitted linear trend."
    )

```

```
elif a1_sss > a1_gsis:
```

```
    print(  
        "SSS has a larger regression slope than GSIS."  
    )
```

```
    print(  
        "This means SSS contributions increase faster "  
        "per year based on the fitted linear trend."  
    )
```

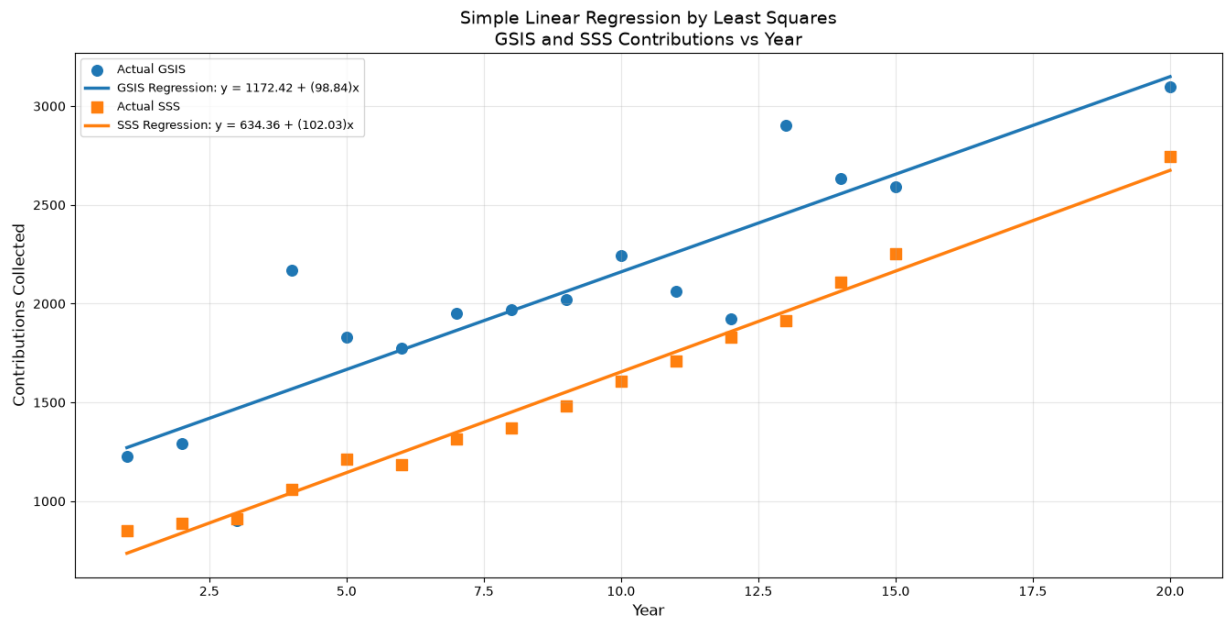
```
else:
```

```
    print(  
        "GSIS and SSS have equal regression slopes."  
    )
```

```
print()
```

```
print("=====")
```

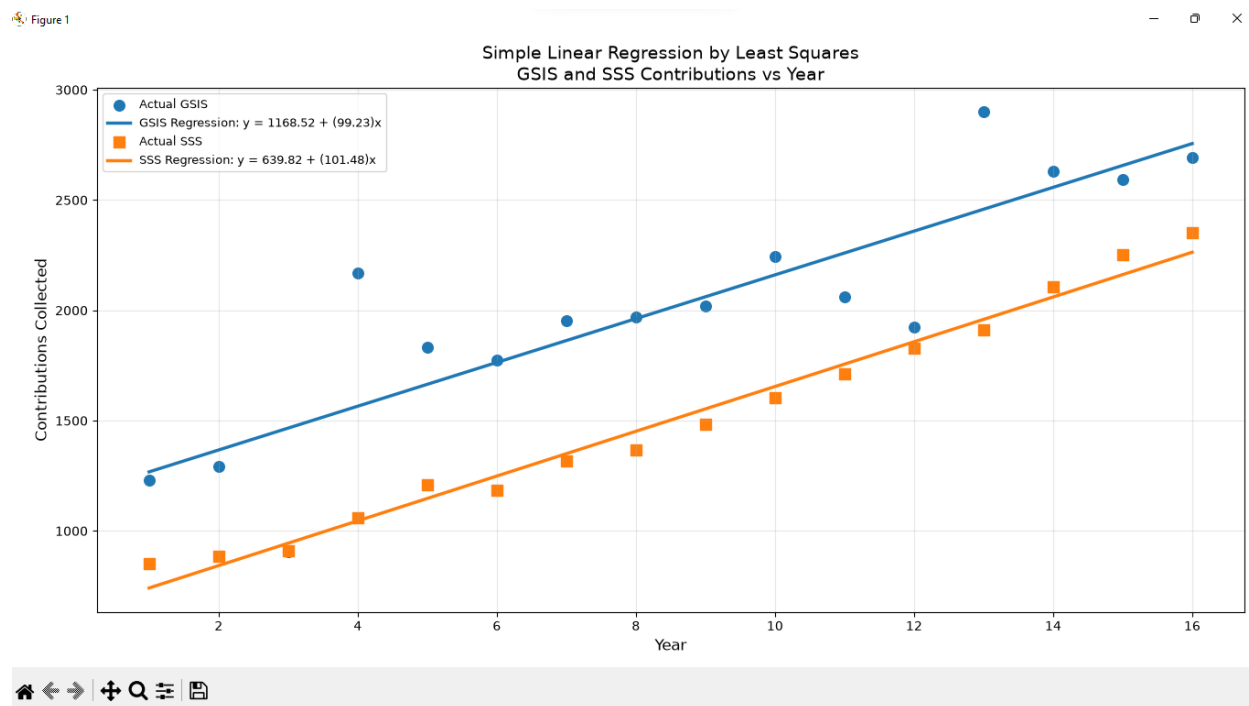
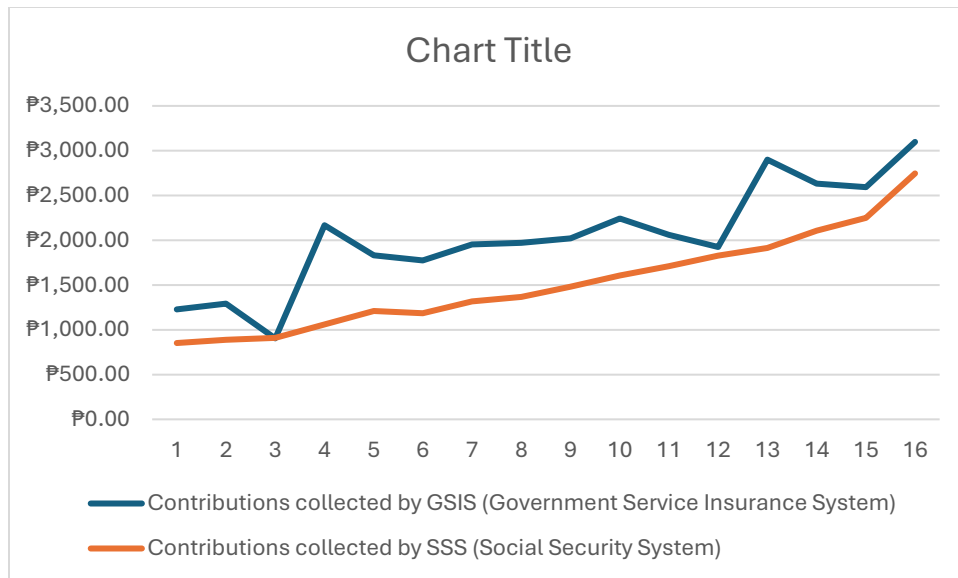
Figure 1



🏠 ⬅ ➡ 🔍 📄

Result	GSIS	SSS
Regression equation $\hat{y} = 1158.259 + 101.037x$	$\hat{y} = 653.999 + 98.982x$	
Intercept a_0	1158.259	653.999
Slope a_1	101.037	98.982
$S_r(\text{SSE})$	1,165,181.300	52,263.130
r^2	0.7104	0.9813
Standard error $s_{y/x}$	299.381	63.405

With these datasets we clearly see that both GSIS and SSS contributions is increasing over time, but SSS is far more consistent in terms of linear line of regression. This claim further proved by the coefficient of determination or r^2 . The GSIS model also shows a positive trend, but its lower r^2 and superb standard error indicate considerably greater variation around the fitted line.



Prediction is significant because if it might observe to come to life using the existing data we calculate earlier.